Big Data Assignment 2

1 Methodology

The assignment asks to explain the methodology and to show a demonstration with fullscreen screenshots for indexing and search. It also asks to show indexing of 100 documents. In my run, the project loaded 978 documents, so the demonstration shows more than the required minimum.

1.1 General design

The repository implements a simple search engine using Docker, Hadoop, Spark, and Cassandra. I used Docker Compose because it gives the same environment for all parts of the project. The file `docker-compose.yml` starts three main services:

- `cluster-master`, which is the Hadoop and Spark master container.
- `cluster-slave-1`, which is the Hadoop worker container.
- `cassandra-server`, which stores the final index for fast query lookup.

The main application folder is `/app` inside the container. It is mounted from the local `app` folder. The entry point is `app/app.sh`. This script starts Hadoop and YARN services, installs Python dependencies, prepares the data in HDFS, creates the index, stores it in Cassandra, and runs one example search query.

1.2 Data preparation

The input documents are stored in `app/data` as text files. The script `prepare_data.sh` checks this folder. If there are not enough valid text files, it can create documents from `a.parquet` by using `prepare_data.py`. Then it uploads the text files to HDFS under `/data`.

After that, `prepare_data.py` builds the input for indexing. It reads the text files with Spark, parses the file name into document id and title, normalizes the text, and writes records to `/input/data`. Each record has this form:

```
doc_id    title    text
```

This format is useful for the MapReduce indexing step because each mapper can read one document with its id, title, and content.

1.3 Indexing pipeline

The indexing is started by:

```
bash index.sh
```

The script `index.sh` runs `create_index.sh` and then `store_index.sh`. The first script creates the index in HDFS by using Hadoop Streaming. There are three MapReduce jobs.

The first MapReduce job reads `/input/data`. The mapper `mapper1.py` tokenizes text with the regular expression `[a-z0-9]+`. It lowercases text and emits document length records and term records. The reducer `reducer1.py` aggregates the term frequency for each pair of term and document. It also keeps document metadata with title and length. Its output is written to `/indexer/stats`.

The second MapReduce job builds the inverted index. The mapper `mapper2.py` changes term records into key-value pairs where the key is the term and the value is `doc_id:tf`. The reducer `reducer2.py` groups all postings for a term. It outputs two types of lines: `VOCAB` lines with document frequency and `INDEX` lines with postings. This output is written to `/indexer/index`.

The third MapReduce job computes corpus statistics. The mapper `mapper3.py` takes document length records, and the reducer `reducer3.py` counts documents and computes average document length. This output is written to `/indexer/corpus`. These values are needed later for BM25 ranking.

1.4 Storage in Cassandra

After the HDFS index is created, `store_index.sh` runs `app.py`. This program reads the files from HDFS and stores them in Cassandra. It creates the keyspace `search_engine` and four tables:

- `documents`: stores document id, title, and document length.
- `vocabulary`: stores each term and its document frequency.
- `inverted_index`: stores each term, document frequency, and postings list.
- `corpus_stats`: stores corpus size and average document length.

I used Cassandra here because the search query needs fast lookup by term. The inverted index is stored with the term as a primary key, so the query program can request only the terms that appear in the user query.

1.5 Search and ranking

Search is started by:

```
bash search.sh "history of science"
```

The script `search.sh` submits `query.py` to Spark on YARN. The query program reads the query from command line arguments, tokenizes it with the same regular expression as the indexer, and then loads needed rows from Cassandra.

The ranking function is BM25. In the code, the parameters are:

```
K1 = 1.0
B = 0.75
```

For each query term, the program uses document frequency from `vocabulary`, postings from `inverted_index`, and document length from `documents`. It broadcasts metadata in Spark, computes partial BM25 scores in an RDD, sums scores by document id, and returns the top 10 documents. The program prints only document id and title, not the score.

One important design detail is that the project uses a simple tokenizer only. It does not remove stop words and does not use stemming. For example, in the query `history of science`, the word `of` is also a query term. BM25 down-weights very common terms by IDF, but they are still part of the score.

1.6 Bonus task: adding one document to the index

The optional bonus task is implemented in `add_to_index.sh`. The user can pass a local text file with the required naming format `<doc_id>_<doc_title>.txt`. The shell script validates the argument, calls `add_to_index.py`, and then stores the raw file in HDFS under `/data`. It also writes one prepared TSV record to `/input/data`, so a future full reindex can include the new document as well.

The incremental Cassandra update is done without rebuilding the whole MapReduce index. The Python helper parses the document id and title from the file name, tokenizes the text with the same tokenizer used by `mapper1.py`, computes term frequencies for this document, and updates these tables:

- `documents`: inserts document id, title, and document length.
- `vocabulary`: increases the document frequency for terms that appear in the new document, or inserts new terms.
- `inverted_index`: appends the new `doc_id:tf` posting to each affected term.
- `corpus_stats`: updates total document count and average document length.

The script rejects duplicate document ids, because replacing an existing document would require subtracting the previous document's term frequencies from the index first. The intended use case for this bonus task is adding a new document after `index.sh` has already created the initial index.

2 Demonstration

2.1 How to run the project

To run the full repository, Docker and Docker Compose must be installed. From the project root, run:

```
docker compose up
```

This starts the Hadoop master, one Hadoop worker, and Cassandra. The master container runs the full flow from `app.sh`.

For manual checks, open a shell in the master container:

```
docker exec -it cluster-master bash
cd /app
```

Then run indexing again:

```
bash index.sh
```

Run a search query:

```
bash search.sh "history of science"
```

The README also gives two more example queries that can be used for additional testing:

```
bash search.sh "christmas story"
bash search.sh "computer network"
```

To check HDFS output manually, use:

```
hdfs dfs -ls /indexer
hdfs dfs -cat /indexer/corpus/part-00000
hdfs dfs -cat /indexer/index/part-00000 | head
hdfs dfs -ls /input/data
hdfs dfs -cat /input/data/part-00000 | head
```

Run the bonus incremental indexer for one local document:

```
bash add_to_index.sh /app/data/12345_New_Document.txt
```

The file must exist in the container and must follow the required assignment naming format. A successful run prints the added document id and title, the tokenized document length, the number of unique updated terms, the new corpus document count, and the new average document length. After this command, the document is available through the Cassandra-backed search index without rerunning all MapReduce jobs.

For example, the expected output has this form:

```
Added document: 12345    New Document
Document length: 12
Updated unique terms: 10
New corpus document count: 979
New average document length: 574.707865
```

To verify the result, I can search for a unique word from the added document. If the incremental update worked, `search.sh` returns the new document id and title in the top results.

2.2 Bonus demonstration

For the bonus check, I created a small document with a unique token and added it to the index:

```
root@cluster-master:/app# cat > /tmp/999999_Bonus_Test_Document.txt <<'EOF'
uniquebonusword cassandra hadoop bm25 incremental indexing test document
EOF

root@cluster-master:/app# bash add_to_index.sh /tmp/999999_Bonus_Test_Document.txt
Added document: 999999  Bonus Test Document
Document length: 8
```

```
Updated unique terms: 8
New corpus document count: 979
New average document length: 574.703779
```

This output shows that the new document was parsed correctly, its 8 tokens were indexed, and the Cassandra corpus statistics were updated from 978 to 979 documents. Then I searched for the unique token:

```
root@cluster-master:/app# bash search.sh "uniquebonusword"
...
999999  Bonus Test Document
```

The Spark command prints many YARN and Spark log lines before the final result. The important line is `999999 Bonus Test Document`, which confirms that the incremental Cassandra index update is visible to the BM25 search application.

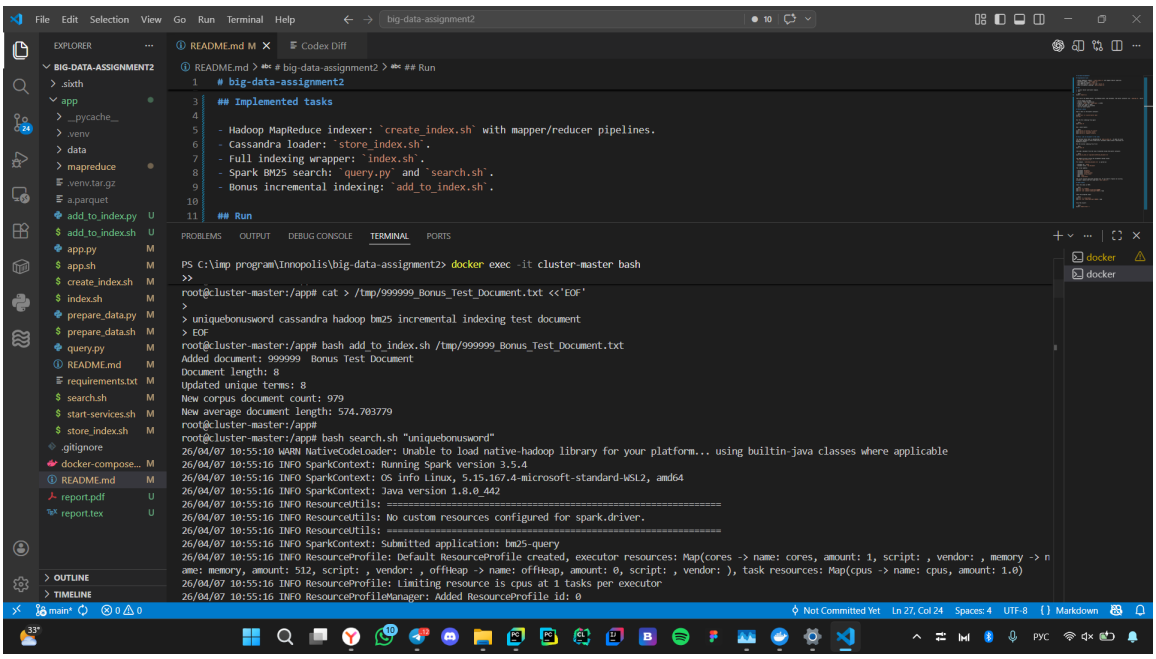

Figure 1: testing the bonus incremental indexing task and searching for the added document.

2.3 Screenshots of running the repository

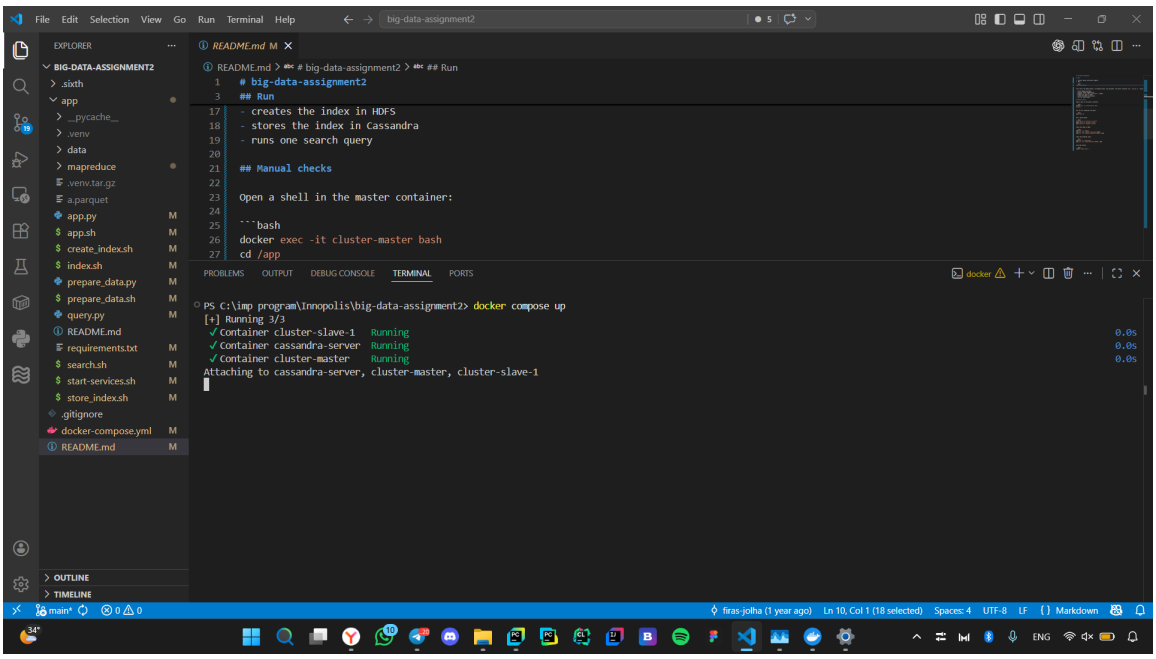

Figure 2: starting the repository with `docker compose up`.

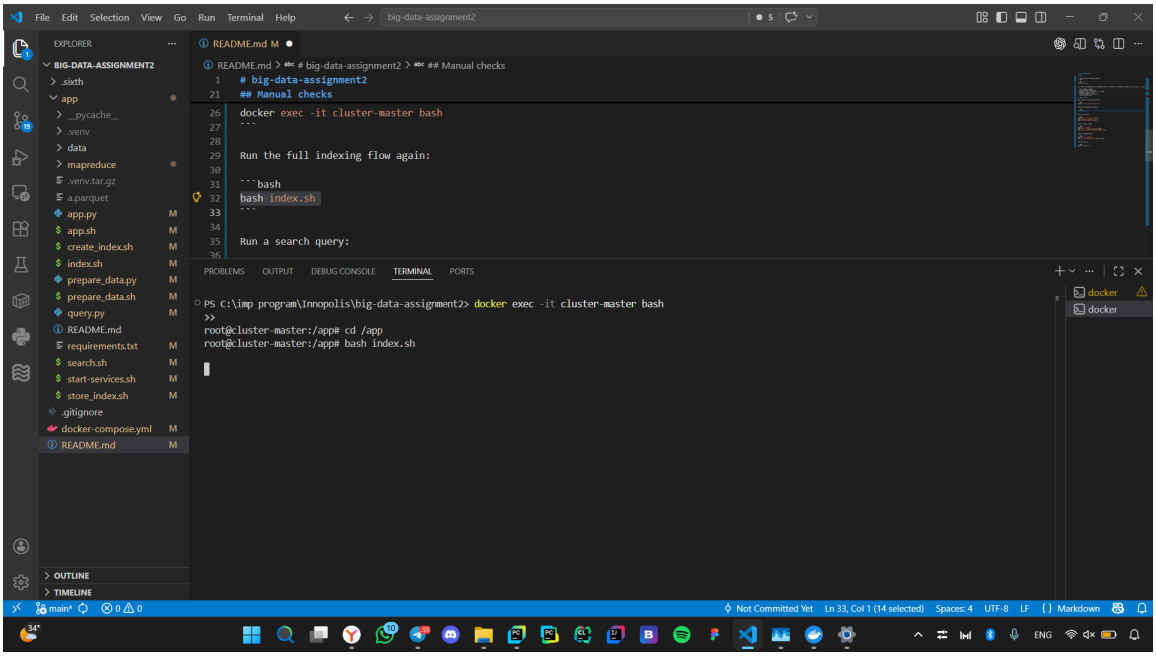

Figure 3: entering the master container and starting `bash index.sh`.

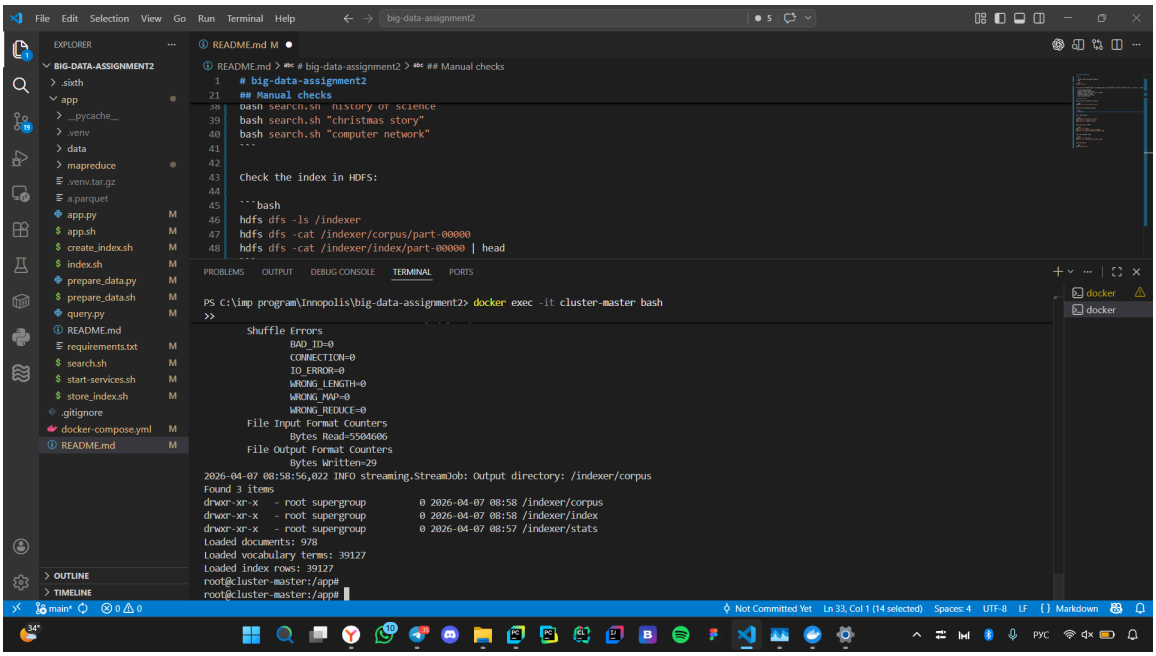

Figure 4: indexing process after running `bash index.sh`.

The important part of the indexing output is shown below. The full Hadoop logs are long, so I show the lines that prove the three MapReduce jobs finished successfully and the index was loaded into Cassandra.

```
root@cluster-master:/app# bash index.sh
...
2026-04-07 08:57:38,210 INFO mapreduce.Job: Job job_1775547385312_0008 completed
    successfully
2026-04-07 08:57:38,602 INFO streaming.StreamJob: Output directory: /indexer/stats
...
2026-04-07 08:58:19,148 INFO mapreduce.Job: Job job_1775547385312_0009 completed
    successfully
2026-04-07 08:58:19,358 INFO streaming.StreamJob: Output directory: /indexer/index
...
2026-04-07 08:58:55,750 INFO mapreduce.Job: Job job_1775547385312_0010 completed
    successfully
2026-04-07 08:58:56,022 INFO streaming.StreamJob: Output directory: /indexer/corpus
Found 3 items
drwxr-xr-x   - root supergroup          0 2026-04-07 08:58 /indexer/corpus
drwxr-xr-x   - root supergroup          0 2026-04-07 08:58 /indexer/index
drwxr-xr-x   - root supergroup          0 2026-04-07 08:57 /indexer/stats
Loaded documents: 978
Loaded vocabulary terms: 39127
Loaded index rows: 39127
```

This output is the main evidence that the index was built. There are three HDFS output folders: `/indexer/stats`, `/indexer/index`, and `/indexer/corpus`. Also, Cassandra loading reports 978 loaded documents, 39127 vocabulary terms, and 39127 index rows.

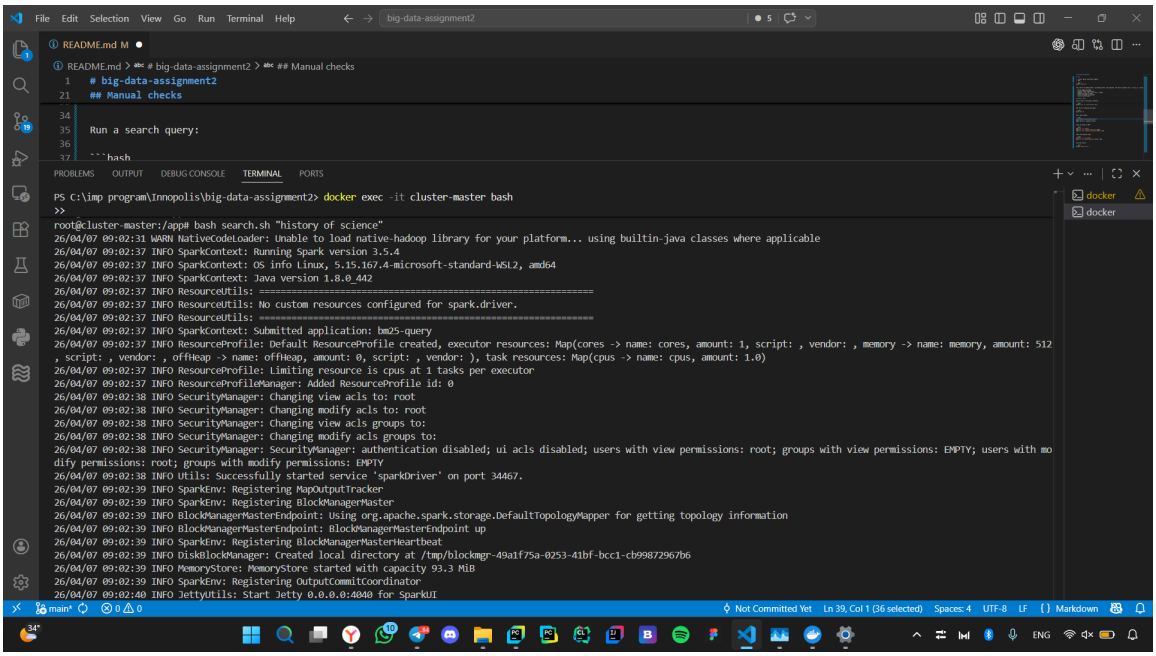

Figure 5: running `bash search.sh "history of science"`.

The useful part of the search output is:

```
root@cluster-master:/app# bash search.sh "history of science"
...
69979031        A History of Science, Technology, and Philosophy in the 16th and 17
    thCenturies
60121915        A Brief History of Everyone Who Ever Lived
2216485         A History of Knowledge
69751062        A Fortune for Kregen
73922368        A Honeymoon in Space
40541172        A History of All Nations from the Earliest Times
67078438        A History of Nomadic Behavior
8191051         A History of the Life and Voyages of Christopher Columbus
21141177        A Key into the Language of America
32151667        A Gnome There Was
```

The first result is very relevant by title because it contains `History` and `Science`. The second and third results also look relevant because they contain `History`. Some lower results are less clear from the printed title, for example `A Fortune for Kregen`. This does not mean that the result is wrong, because the search engine ranks full document text, but it prints only the document title. Also, there is no stop word removal, so common words from the query can still affect the result, even if BM25 gives them lower weight.

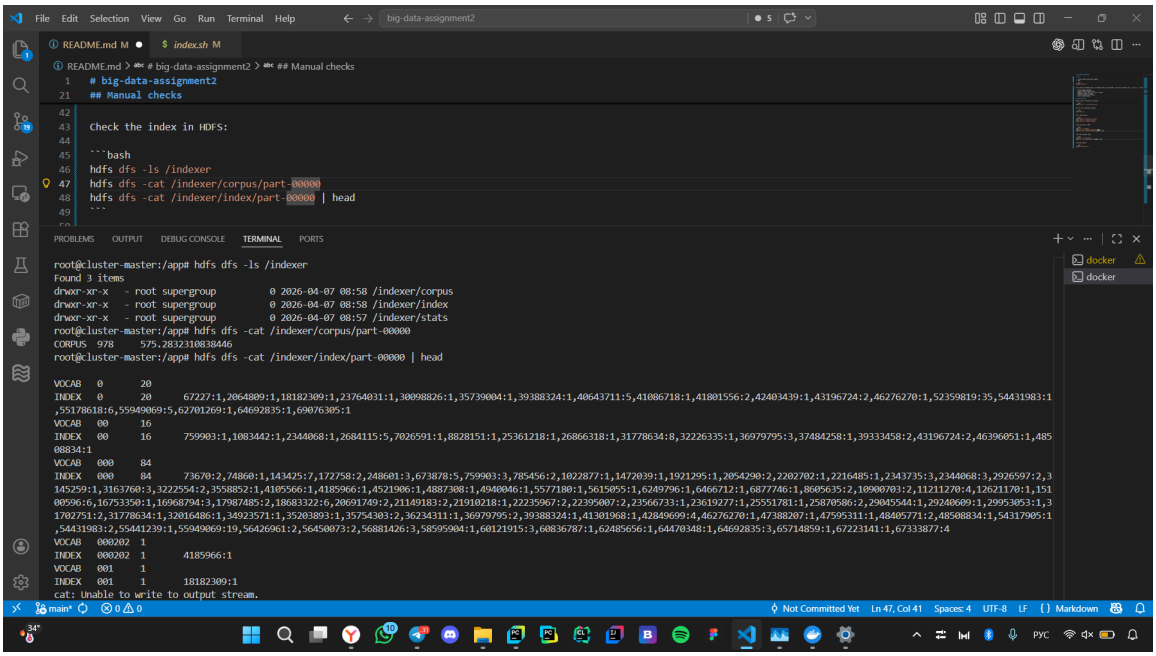

Figure 6: checking the index output in HDFS.

The HDFS check shows the created index folders and sample index records:

```
root@cluster-master:/app# hdfs dfs -ls /indexer
Found 3 items
drwxr-xr-x   - root supergroup          0 2026-04-07 08:58 /indexer/corpus
drwxr-xr-x   - root supergroup          0 2026-04-07 08:58 /indexer/index
drwxr-xr-x   - root supergroup          0 2026-04-07 08:57 /indexer/stats

root@cluster-master:/app# hdfs dfs -cat /indexer/corpus/part-00000
CORPUS   978     575.2832310838446

root@cluster-master:/app# hdfs dfs -cat /indexer/index/part-00000 | head
VOCAB    0       20
INDEX    0       20      67227:1,2064809:1,18182309:1,23764031:1,30098826:1,...
VOCAB    00      16
INDEX    00      16      759903:1,1083442:1,2344068:1,2684115:5,7026591:1,...
VOCAB    000     84
INDEX    000     84      73670:2,74860:1,143425:7,172758:2,248601:3,...
cat: Unable to write to output stream.
```

Here `CORPUS` means that the corpus has 978 documents and average document length is about 575.28 tokens. The `VOCAB` lines show terms and document frequency. The `INDEX` lines show postings in the form `doc_id:term_frequency`. The final `cat` warning appears after using `head`; it is not the main result of the command, because the index lines were already printed.

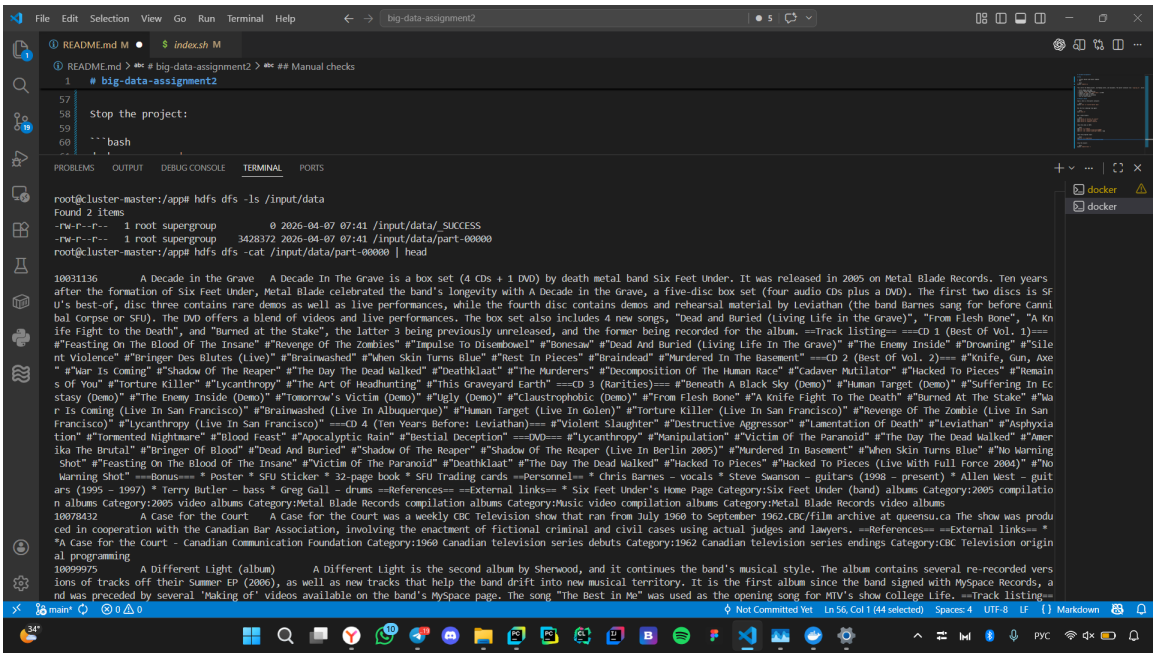

Figure 7: checking the prepared input in HDFS.

This check is important because the indexer reads from `/input/data`. The commands from the README are:

```
hdfs dfs -ls /input/data
hdfs dfs -cat /input/data/part-00000 | head
```

If this path exists and contains records, then the MapReduce indexer has valid input.

2.4 Reflection

The project works as a simple distributed search engine. Hadoop MapReduce is good for building the index because it processes many documents in batch mode. Cassandra is useful after indexing because query-time lookup is by term. Spark is used during search to compute BM25 scores and take the top 10 results.

From the result for `history of science`, the top document is expected and easy to understand. The ranking is not perfect to read from the output because the program prints only document id and title. For a better search demonstration, I would also print the BM25 score and maybe a short text snippet. I would also consider stop word removal, because words like `of` are not very useful for the user query.